

Autonomous Accounts Payable & Invoice Exception Handler

Production-Oriented Architecture — Expanded Edition

Domain	Pattern	Target Runtime
Enterprise ERP & Financial Automation	Multi-Agent / Deterministic Validation / HITL	Python / LangGraph / FastAPI / PostgreSQL

Core objective: Convert incoming supplier invoices into validated, auditable, policy-compliant ERP actions while keeping financial decisions deterministic where possible and routing ambiguity or risk to humans.

This version expands the original four-stage architecture with invoice ingestion, duplicate detection, risk scoring, policy management, a human approval dashboard, ERP execution controls, and an AgentOps/evaluation layer.

Architecture principle

Use LLMs for perception, classification, explanation, and ambiguity resolution; use deterministic code/SQL for financial calculations, policy enforcement, idempotency, and transaction safety.

1. Executive Summary & Core Objective

The Autonomous Accounts Payable & Invoice Exception Handler is an agentic enterprise workflow for processing supplier invoices from ingestion through validation, exception resolution, human approval, and ERP posting. The system is designed around a hybrid model: AI handles unstructured document understanding and exception reasoning, while deterministic services enforce financial correctness.

Primary business outcomes

- Reduce manual invoice entry and exception handling.
- Increase straight-through processing for clean invoices.
- Detect duplicate, anomalous, and potentially fraudulent invoices.
- Apply configurable tolerance and approval policies.
- Provide auditable human approval for sensitive transactions.
- Prevent duplicate or unsafe ERP postings through idempotent execution.
- Measure extraction quality, agent decisions, latency, cost, and human overrides.

Design rule: No LLM output directly posts money. Every payment-impacting action passes through deterministic validation, policy, authorization, and execution controls.

2. End-to-End Procure-to-Pay Workflow

The expanded workflow begins before document extraction and continues after an invoice decision so the system covers the complete operational lifecycle.

Invoice Source → Ingestion → Vision Extraction → Deterministic Validation → Duplicate Detection → Risk/Fraud Scoring → Exception Reasoning → Policy Engine → Auto-Resolution or HITL → ERP Execution → Audit/Event Log → AgentOps & Evaluation

Stage	Purpose	Primary output
1. Invoice Ingestion	Receive invoice from email, upload, API, or batch source.	Normalized document + metadata
2. Vision Extraction	Convert unstructured PDF/image content into structured invoice	Validated extraction payload
3. Deterministic Validation	Perform PO/GRN/tax/math/tolerance checks using code and SQL	Validation scorecard
4. Duplicate Detection	Detect exact and fuzzy duplicate invoices.	Duplicate risk + match evidence
5. Exception Reasoning	Classify discrepancies and propose resolution.	Exception category + action plan
6. Risk/Fraud Scoring	Assess vendor, transaction, bank, and invoice anomalies.	Risk score + reasons
7. Policy Engine	Apply configurable business and approval policies.	Decision / escalation route
8. HITL Dashboard	Allow authorized users to approve, reject, or request correction.	Human decision
9. ERP Execution	Post approved transactions safely and idempotently.	ERP transaction result
10. AgentOps + Evaluation	Measure quality, cost, reliability, and regressions.	Metrics + evaluation reports

3. Ground-Truth Business Documents & Data Model

Document / Entity	Role	Important fields
Purchase Order (PO)	Approved commercial commitment.	po_number, vendor_id, item_id, agreed_unit_price, approved_qty
Goods Receipt Note (GRN)	Physical receipt evidence.	grn_number, po_number, received_qty, damaged_qty, received_date
Vendor Invoice	Supplier payment request.	invoice_number, po_number, billed_qty, billed_unit_price, tax, total_due

Document / Entity	Role	Important fields
Vendor Contract	Commercial terms and allowed charges.	vendor_id, pricing_terms, shipping_terms, effective_dates
Vendor Profile	Historical behavior and risk context.	vendor_id, history, bank details, anomaly statistics
Policy	Configurable financial decision rules.	rule_id, threshold, tolerance, approver_role, action
Audit Event	Immutable workflow evidence.	event_id, invoice_id, actor, timestamp, action, result

The PO, GRN, invoice, and contract provide the core business evidence. Vendor history, policies, and audit events extend that evidence into risk and governance.

4. Core Modules & Responsibilities

4.1 Invoice Ingestion

- Accept invoices through email inboxes, REST APIs, file uploads, and batch imports.
- Normalize metadata, store the original document, assign a correlation ID, and calculate a document hash.
- Perform basic file validation and route unsupported or corrupted files to a dead-letter path.

4.2 Vision Extraction

- Process multi-page invoices and scanned documents using a vision-capable model.
- Extract supplier, invoice number, PO reference, line items, quantities, prices, taxes, totals, currency, and charges.
- Return structured Pydantic data with field-level confidence and source/page references where possible.
- Reject malformed or low-confidence payloads instead of silently passing them downstream.

4.3 Deterministic Validation Engine

- Retrieve PO and GRN records using controlled database tools.
- Perform line-level quantity, price, subtotal, tax, total, and tolerance calculations.
- Validate contract constraints and allowed charges where contract data is available.
- Keep mathematical and financial validation outside the LLM.

5. Matching, Tolerance & Validation Rules

The original strict three-way match is retained as the baseline, but the production system introduces configurable tolerances rather than assuming every variance requires rejection.

Baseline: Invoice PO Reference == PO Number AND Invoice Quantity == GRN Quantity AND Invoice Price == PO Price

Example tolerance policy

Rule	Example policy	Outcome
Price variance	±2%	Within tolerance → pass
Quantity variance	±5%	Within tolerance → pass or controlled exception
Tax variance	≤ configured absolute threshold	Small rounding differences accepted
Shipping	Only approved contract/PO charges	Otherwise exception
Approval amount	> configured threshold	Mandatory human approval

6. Duplicate Detection

Duplicate detection protects the AP workflow from repeated uploads, supplier resubmissions, or semantically identical invoices with minor formatting changes.

- Exact document hash detection.
- Exact invoice-number + vendor matching.
- Fuzzy invoice-number normalization such as INV-8821 vs INV8821.
- Similarity checks using vendor, amount, PO, date, and line-item features.
- Route uncertain duplicates to HITL rather than automatically rejecting them.

Example: a document that differs visually but matches the same vendor, invoice number, amount, and PO should receive a high duplicate score.

7. Risk & Fraud Scoring

A separate risk layer evaluates whether an otherwise processable invoice has suspicious characteristics. This is not a claim that the system can prove fraud; it is a prioritization mechanism for review.

Signal	Example
Vendor anomaly	Invoice materially exceeds historical vendor amounts.
Price anomaly	Unit price is far outside historical or contracted range.
Payment-change risk	Bank account changed shortly before a large invoice.
Duplicate risk	Invoice closely resembles a previously processed invoice.
New vendor risk	First invoice from a vendor with limited history.
Behavioral anomaly	Unusual frequency, timing, amount, or charge pattern.

Illustrative risk output: Overall risk score 0.91, with duplicate risk 0.92, bank-change risk 0.98, vendor anomaly 0.84, and price anomaly 0.71. The score should be explainable through the contributing signals.

8. Exception Reasoning Agent

The Exception Reasoning Agent receives deterministic discrepancy evidence and converts it into a structured explanation and recommended action. It should not invent financial facts that are not present in the evidence.

Responsibilities

- Classify exceptions: price, quantity, tax, shipping, duplicate, contract, vendor, or other.
- Explain the discrepancy using PO, GRN, invoice, contract, and historical evidence.
- Recommend actions such as approve, request corrected invoice, request credit note, or escalate.
- Produce a structured action plan for the policy engine and HITL interface.

9. Policy Engine

Business policy should not be buried inside agent prompts. A dedicated policy engine makes approval thresholds, tolerances, vendor restrictions, and escalation rules configurable.

Illustrative policy flow

Risk < 0.30 + all validations pass → Auto-process
Risk 0.30-0.70 or controlled variance → Review / conditional action
Risk > 0.70 or sensitive condition → Mandatory HITL
Any failed safety/idempotency check → Block execution

- Policies stored in PostgreSQL/configuration rather than hardcoded in agent logic.
- Version every policy so historical decisions can be reconstructed.

- Record which policy version produced each decision.

10. Human-in-the-Loop Dashboard

The HITL dashboard is the operational control surface for exceptions and high-risk transactions.

Dashboard view	Information
Overview	Invoices processed, auto-approved, exceptions, pending reviews.
Exception queue	Invoice, vendor, amount, exception type, risk, age, SLA.
Invoice detail	Original document, extracted fields, PO, GRN, contract, calculations.
Agent evidence	Reasons, tools used, confidence, discrepancy evidence.
Decision controls	Approve, reject, request correction, escalate.
Audit history	Every state transition and human action.

Human actions should be authenticated, role-based, timestamped, and written to the audit log. The interface should make the evidence behind an agent recommendation visible before approval.

11. ERP Execution Layer

The ERP layer is intentionally separated from reasoning. Approved actions are converted into ERP-specific transactions through an adapter interface.

Core workflow: Policy Approval → Authorization Check → Idempotency Check → ERP Adapter → Transaction Verification → Audit Event

- Support a mock ERP first, followed by adapters for real enterprise systems.
- Use idempotency keys so retries cannot create duplicate financial transactions.
- Separate authorization from recommendation: an agent can recommend, but execution requires policy permission.
- Handle ERP timeouts and partial failures using retry and reconciliation workflows.

12. Auditability, Reliability & Security

- Persist immutable workflow events for every material decision.
- Store agent version, prompt/configuration version, model, tool calls, and decision evidence.
- Use correlation IDs across ingestion, LangGraph state, database operations, and ERP execution.
- Implement retries, exponential backoff, dead-letter handling, and recovery from interrupted workflows.
- Enforce RBAC for AP operators, managers, finance administrators, and system services.
- Protect invoice and vendor data through encryption, secrets management, and least-privilege database access.
- Never allow arbitrary LLM-generated SQL or unrestricted ERP commands.

13. AgentOps & Evaluation Platform

The system should measure whether the automation is actually improving AP operations rather than assuming that an agent is successful because a workflow completed.

Metric group	Examples
Extraction quality	Field accuracy, confidence, missing-field rate.
Decision quality	Correct approval/rejection, exception classification accuracy.
Operational	Processing time, throughput, retry rate, failure rate.

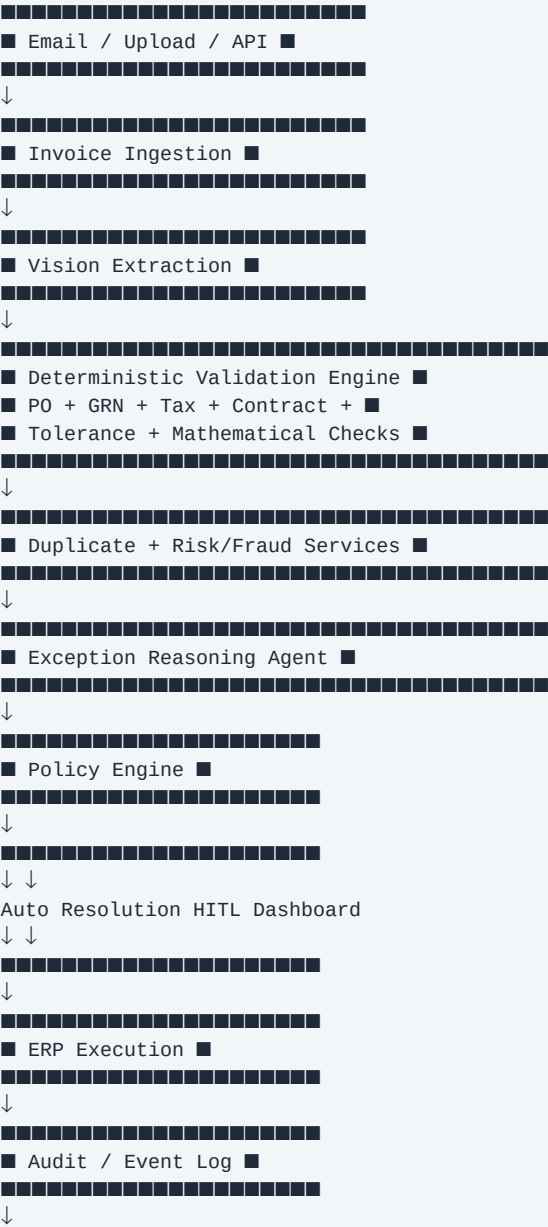
Metric group	Examples
AI economics	Tokens, model latency, LLM cost per invoice.
Human interaction	HITL rate, override rate, approval time.
Business	STP rate, exception rate, duplicate prevention, prevented overpayment.

Evaluation dataset

- Maintain a labeled invoice/PO/GRN benchmark set with known ground truth.
- Run regression tests when changing models, prompts, extraction schemas, or policies.
- Compare extraction and decision accuracy across model versions.
- Track false positives and false negatives for duplicate and risk detection.
- Store evaluation results as versioned experiment runs.

14. Reference System Architecture

The recommended architecture separates perception, deterministic controls, reasoning, governance, execution, and observability.



15. Concrete Operational Example

Scenario: TechCorp issues PO #9901 for 10 laptops at \$1,000/unit. The warehouse receives 9 units because one is damaged. Supplier invoice INV-8821 bills 10 units at \$1,050/unit plus an unexpected \$200 shipping fee.

Stage	System behavior
Ingestion	Receives invoice PDF, stores original, assigns correlation ID and document hash.
Extraction	Extracts PO 9901, quantity 10, unit price \$1,050, shipping \$200.
Validation	Finds 10 billed vs 9 received; \$1,050 vs \$1,000 PO price; unapproved shipping.
Duplicate	Checks invoice number, vendor, amount, PO, hash, and similarity against history.
Risk	Scores unusual price/shipping and any relevant vendor/payment anomalies.
Reasoning	Classifies quantity, price, and unapproved-fee exceptions and proposes a resolution.
Policy	Determines which actions can be automated and which require manager approval.
HITL	Manager reviews evidence and approves/rejects the sensitive shipping decision if required.
ERP	Only after authorization and idempotency checks, posts the permitted transaction.
AuditOps	Records every decision and measures processing time, cost, outcome, and overrides.

The quantity variance can be calculated as 10 billed minus 9 received. The price variance can be calculated against the accepted quantity. The shipping charge should be handled according to the configured contract/policy rather than assumed to be valid or invalid.

16. Implementation Roadmap

Phase	Build
Phase 1 — Foundation	PostgreSQL schema, object storage, invoice ingestion, Pydantic models, correlation IDs.
Phase 2 — Extraction	Vision extraction, structured output validation, confidence handling.
Phase 3 — Validation	PO/GRN matching, tolerance engine, tax/math validation, contract checks.
Phase 4 — Risk	Duplicate detection, vendor anomaly features, risk scoring.
Phase 5 — Agent	Exception reasoning, evidence-based action plans, LangGraph orchestration.
Phase 6 — Governance	Policy engine, HITL dashboard, RBAC, audit trail.
Phase 7 — Execution	Mock ERP adapter, idempotent transaction posting, retries/reconciliation.
Phase 8 — AgentOps	Tracing, metrics, cost tracking, benchmark dataset, regression evaluation.
Phase 9 — Production Hardening	Security, load testing, failure injection, monitoring, deployment.

17. Recommended Technology Stack

Layer	Technology
Orchestration	Python + LangGraph

Layer	Technology
API	FastAPI
Data validation	Pydantic
Database	PostgreSQL
Vector / retrieval	Pgvector where justified by the retrieval use case
State / caching	Redis
Document storage	Object storage
Agent tools	MCP or controlled internal tool interfaces
Vision	Vision-capable LLM / document model
Dashboard	React-based web application
Observability	OpenTelemetry-compatible tracing + metrics/logging
Evaluation	Versioned benchmark dataset + automated regression runs
Deployment	Docker; production orchestration as needed

18. Definition of Done / Success Criteria

- A valid invoice can travel from ingestion to ERP posting without manual intervention when all policies pass.
- An invoice with a price/quantity/tax/contract discrepancy is explained and routed correctly.
- A repeated invoice is detected without creating a duplicate payment.
- High-risk invoices are blocked or routed to authorized human reviewers.
- Every financial action is reproducible from the audit trail.
- ERP retries cannot create duplicate transactions.
- Model or prompt changes can be evaluated against a fixed benchmark.
- The system exposes measurable STP, accuracy, latency, cost, failure, and human-override metrics.
- The LLM cannot bypass deterministic validation, policy, authorization, or execution controls.

Final architectural principle: The goal is not to maximize the number of agents. The goal is to build a reliable autonomous financial workflow in which agents are used where reasoning is valuable and deterministic software controls everything that must be exact.